Databricks Deployment - KT Notes

Simple handover notes based on the provided working notes

1. Overview

We mainly handle three types of Databricks deployment:

- Normal notebook / DDL deployment
- Wheel file deployment
- DLT (Delta Live Tables) deployment

The deployment flow depends on the environment (ITG or Production) and on whether the developer wants only code deployment or also wants to execute the notebook on a cluster.

2. Normal Notebook / DDL Deployment

Basic flow

1. Developer raises a PR from the develop branch to the release ITG branch for an ITG deployment.

2. If the developer wants the notebook to run on the cluster, the PR title should contain **EXECUTE=YES**.

3. The pipeline installs the Databricks CLI.

4. The pipeline adds the required host file entry based on the target environment.

5. The ITG repos folder is refreshed. After the PR is approved, code from the release ITG branch moves to the Databricks repo path.

6. If EXECUTE=YES is present, the notebook execution task runs. The notebook is attached to the cluster and triggered through the pipeline.

Why host file entries are needed

Databricks cannot be accessed directly from outside the VDI because private endpoints are enabled by the platform team. Because of this, the correct host file entry is required. ITG deployment uses ITG host entries and Production deployment uses Production host entries.

Notebook execution task - main steps

- Import all required modules.
- Perform GitHub authentication.
- Get the PR author.
- Clone the DevOps pipeline repository.
- Copy required files / paths from secure files to the build source directory.
- Update the job cluster values and trigger the job cluster.

Files used in the normal DDL deployment

File	Purpose
config.py	Contains environment-specific variables and server / storage details.
library.txt	Contains the path used for the run.py file.

run.py	Contains or refers to the DDL scripts that need to run.

Developer flow for DDL code

1. Developer writes the code inside the DDL folder.

2. Developer specifies the required path in run.py.

3. The deployment uses the environment details from config.py and the paths/files referenced by the other deployment files.

Storage account variables mentioned in the notes

- INBOUND_STORAGE_ACT -> use **INBSTGACCT**
- FDL_STORAGE_ACC -> use **FDLSTGACCT**
- CONSUMPTION_STORAGE_ACT -> use **CONSSTGACCT**

These variables are used by developers in their code. The notes also mention that, on the DevOps cluster, only the create script is normally allowed to run.

3. Wheel File Deployment

A wheel file is created when developers release a new version of Python code. The new version is built and then deployed to the required storage account.

1. Use Python 3.8.

2. Load Python dependencies.

3. Run the required Python script.

4. Build the Python wheel file from the code sent through GitHub PR.

5. Store the wheel file as a CI artifact.

6. The CD flow picks the artifact from CI, takes the wheel file, and deploys it to the required storage account / server.

7. After deployment, the wheel file is manually attached to the cluster.

4. Changes in Existing Databricks Environment

For an existing environment, some changes are handled through requests rather than directly through the deployment pipeline.

- For ITG: raise an **MTI** request.
- For Production: raise an **MTP** request.
- Examples: update a wheel file, change notebook path, add a new parameter, enable notification, or add retries.
- For a newly created cluster with no previous run, the developer raises a SharePoint request to attach the required library, wheel file, and path.
- After the cluster has run even once, further such changes should come through MTI / MTP and the release team.

5. DLT (Delta Live Tables) Deployment - Manual Process

The developer provides the DLT pipeline name and pipeline ID. We first check the pipeline in the Dev environment and then replicate the required settings in ITG.

1. Open the Dev environment and locate the required DLT pipeline.

2. Open Settings and review all configuration.

3. Go to ITG -> DLT Live Tables -> Create Pipeline.

4. Replicate the required details such as pipeline name, Product Edition (Pro), Pipeline Mode (Triggered), repo path, destination, target schema, compute policy, and configuration.

5. The storage location is important because it cannot be modified later. If it is wrong, the pipeline may need to be created again from scratch.

6. The cluster policy is environment-specific, so use the policy for the target environment.

7. For configuration, copy the Dev JSON, replace Dev-specific values with ITG values, and paste the updated configuration into ITG.

8. Save the pipeline.

Permissions in manual DLT creation

- For Dev, confirm the required admin group and user group with the developer.
- For ITG, owner access is given to HVP as noted, while admin and user groups get Can Manage access.

6. DLT Auto Deployment Process

In auto deployment, the pipeline reads the Dev pipeline JSON, makes the required environment changes, and posts the updated configuration to the target environment using API calls.

Initial / setup steps

1. Add entry to the host file.

2. Install Databricks CLI.

3. Clone the DevOps pipeline repository.

4. Run pip install.

5. Generate the SVP token.

6. Run the PowerShell script.

> The notes mention that these initial steps are mostly standard installation / preparation steps and normally do not require value changes.

Get Dev pipeline JSON

The Dev pipeline ID is stored in **dlt_ids_prod.txt**. That ID is used to extract the JSON data of the Dev environment.

Update pipeline JSON

The JSON is modified for the target environment. The notes mention replacing Dev values with ITG or Production values and updating the following items:

- IDH SAP storage value.
- FDL SAP storage value.
- Scope.
- Policy ID - find the target environment policy under Compute -> Policies and replace the Dev policy ID.
- Notebook path - copy the Dev notebook path, replace Dev with ITG / Production, and update the pipeline-specific end part of the path.

Deploy and update permissions

1. Deploy the pipeline into the target environment.

2. Update permissions using the relevant permission JSON, for example permission_prod.json.

Finding Service Principal and Group IDs

- Go to Profile -> Settings -> Identity and Access.
- Open Users, Groups, Service Principals.
- Open the required Service Principal, click Manage, and copy the ID.
- Ask the developer which admin and user entries are required for the DLT pipeline.
- Replace the required IDs in the permission JSON. The same type of process is followed for group names.

Owner note

Only one owner can exist. During pipeline creation, a default owner may already be assigned. The notes say to change that default owner permission to Can View and then set the required owner. The GUID used in the updated pipeline should be for the target environment.

7. Common Issues and Troubleshooting

Conflict resolution: The refresh task may fail. Go to the specific repo, open the branch, and discard the conflicting changes.

EXECUTE=YES missing: If the developer wants to run the notebook on the cluster, EXECUTE=YES must be given. Otherwise execution will not happen.

Permission issue: Platform team checks permission-related issues.

Code issue: Developer checks code-related issues.

Pipeline retry: From the DevOps side, the pipeline can be retriggered. If nothing changed, it will repeat the same process.

External location missing: External locations are created by the platform team. After creation, the DevOps team retriggers the pipeline.

New storage account: If a developer needs a new storage account but no environment variable exists in config.py, DevOps creates the environment variable on the cluster using the value provided by the developer and updates config.py. Path mentioned: Compute -> Advanced -> Environment Variable.